



PDF Download
3477495.3531833.pdf
28 January 2026
Total Citations: 50
Total Downloads: 999

Latest updates: <https://dl.acm.org/doi/10.1145/3477495.3531833>

SHORT-PAPER

An Efficiency Study for SPLADE Models

CARLOS EDUARDO LASSANCE, Naver Labs Corporation, Seongnam, Gyeonggi-do, South Korea

STÉPHANE CLINCHANT, Naver Labs Corporation, Seongnam, Gyeonggi-do, South Korea

Open Access Support provided by:

Naver Labs Corporation

Published: 06 July 2022

[Citation in BibTeX format](#)

SIGIR '22: The 45th International ACM
SIGIR Conference on Research and
Development in Information Retrieval
July 11 - 15, 2022
Madrid, Spain

Conference Sponsors:
SIGIR

An Efficiency Study for SPLADE Models

Carlos Lassance
Naver Labs Europe
Meylan, France
first.lastatnaverlabs.com

Stéphane Clinchant
Naver Labs Europe
Meylan, France
first.lastatnaverlabs.com

ABSTRACT

Latency and efficiency issues are often overlooked when evaluating IR models based on Pretrained Language Models (PLMs) in reason of multiple hardware and software testing scenarios. Nevertheless, efficiency is an important part of such systems and should not be overlooked.

In this paper, we focus on improving the efficiency of the SPLADE model since it has achieved state-of-the-art zero-shot performance and competitive results on TREC collections. SPLADE efficiency can be controlled via a regularization factor, but solely controlling this regularization has been shown to not be efficient enough. In order to reduce the latency gap between SPLADE and traditional retrieval systems, we propose several techniques including L1 regularization for queries, a separation of document/query encoders, a FLOPS-regularized middle-training, and the use of faster query encoders. Our benchmark demonstrates that we can drastically improve the efficiency of these models while increasing the performance metrics on in-domain data. To our knowledge, we propose the first neural models that, under the same computing constraints, *achieve similar latency (less than 4ms difference) as traditional BM25*, while having *similar performance (less than 10% MRR@10 reduction) as the state-of-the-art single-stage neural rankers on in-domain data*.

CCS CONCEPTS

• Information systems → Information retrieval.

KEYWORDS

latency, information retrieval, splade, sparse representations

ACM Reference Format:

Carlos Lassance and Stéphane Clinchant. 2022. An Efficiency Study for SPLADE Models. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22)*, July 11–15, 2022, Madrid, Spain. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3477495.3531833>

1 INTRODUCTION

As search engines process billion of queries every day, efficiency has been a long standing research topic in Information Retrieval (IR). For instance, optimizing an inverted index with compression techniques or adopting an efficient two stage ranking pipeline to

improve performance while maintaining an acceptable latency are commonplace for most search engines. Today, the advances of Pretrained Language Models (PLMs) have challenged the foundations of many ranking systems, which are based on term-based approaches like BM25 for first-stage ranking [38]. Among this new generation of first-stage rankers, there exist two types of models. On one hand, the *dense* retrieval models [16, 23, 37, 46] rely on Approximate Nearest Neighbors (ANN) techniques developed by the Computer Vision community. On the other hand, the *sparse* retrieval models [1, 6, 11, 13, 27, 35, 49] perform matching at the token level and therefore use a traditional inverted index for scoring.

If TREC competitions and other retrieval benchmarks report performance measures such as NDCG@10 and have shown the immense benefit of pretrained language models, the overall picture for latency is less clear. In fact, measuring the latency of these novel models is challenging as there could be multiple testing conditions. For instance, a standard dense bi-encoder may rely on multiple CPUs to perform the search while some systems rely only on a single core and others on multi-core implementations. An advantage of sparse retrieval models is the vast literature [4, 8, 9, 20, 22, 28, 30, 43] in optimizing retrieval with inverted indices. Furthermore, these works achieve impressive mono-cpu retrieval numbers for traditional sparse retrieval models [24], making it simple to improve the scalability of the system (one just needs to add more cpus). This differs vastly from the context of dense retrieval, where multi-threaded (and sometimes even GPU [16]) seems to be the norm. Second, integrating a sparse ranker into an existing system may be less costly compared to the integration of a dense retrieval system. Note that even if the dense systems use a lot more compute (multi-cpu core + GPU vs mono-cpu core), the latency numbers from dense models (e.g. TAS-B has retrieval+inference of 64 ms for 34.3 MRR@10) [16] tend to be equivalent than the one of sparse models (e.g. unicon-T5 has retrieval + inference 36 ms + 45 = 81ms for 35.2 MRR@10) [25]. We include a short comparison on the appendix, but will focus on sparse retrieval for the rest of this work.

In this work, we focus on the SPLADE model as it was shown to be a state-of-the-art sparse retrieval model [10]. Regarding latency, the model was actually optimized for multi-thread retrieval and no numbers were given, until a recent study [25] reported that SPLADE was not well suited to mono-cpu retrieval environment. In this paper we aim at reducing this gap in performance, and thus focus on reducing mono-thread retrieval of SPLADE models under the PISA [29] and ANSERINI [48] frameworks. Our main contribution is the proposal of 4 adaptations to the SPLADE model, namely: i) separation of query and document encoders, ii) change the query regularization to L1, iii) a FLOPS-regularized PLM, and iv) the use of a smaller PLM for query encoding. In order to showcase the utility of our adaptations, we focus on three main research questions:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGIR '22, July 11–15, 2022, Madrid, Spain

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8732-3/22/07...\$15.00
<https://doi.org/10.1145/3477495.3531833>

- RQ1: How does the performance (efficiency and effectiveness) of SPLADE evolve as we introduce each subsequent adaptation?
- RQ2: How do these new SPLADE models compare to the state of the art in terms of in-domain sparse retrieval?
- RQ3: How do these new SPLADE models compare to the state of the art in terms of out-of-domain sparse retrieval?

2 PREVIOUS WORKS

Measuring efficiency of PLM-based retrieval systems: Comparing efficiency metrics between methods is a complicated task. While benchmarking effectiveness is easy because all methods focus on solving the same task, and thus we can find “one” metric that is comparable between all methods, evaluating efficiency is naturally a trade-off (how much impact does it have on the effectiveness metric). Efficiency depends on which type of system it is focused on (multi-thread or mono-thread per query retrieval) and also depends heavily on the machine used to perform the measures [15]. For example, efficient sparse retrieval is a domain in itself, with a rich diversity of methods [4, 8, 9, 20, 22, 28, 30, 43] that have been proposed in order to improve their retrieval times. Note that the “best method” depends on various factors [20, 25] and there is no one method that is better than all the others. In this work, our focus is to propose adaptations to the SPLADE model and not to the retrieval process itself, thus we avoid these open questions by using the document-at-a-time retrieval setup from previous works [24, 25] in order to perform our study. However, we believe that jointly improving the PLM-based model and the retrieval process should lead to even more improved performance.

PLM-based dense retrieval: Recently, IR tasks have been taken by an avalanche of PLM-based dense retrievers [12, 16, 23, 37, 46]. However, due to their dense nature, efficiency studies have been mostly confined on multi-cpu or even GPU-based systems [16] and will therefore not be considered here as they are not comparable. For instance, [16] reports “quick” methods that have less than 70ms in multi-cpu/GPU systems, while we consider models with less than 10ms on mono-cpu settings. Furthermore, dense retrieval models have lower performances on out-of-domain retrieval [41].

PLM-based sparse retrieval: Another way to perform retrieval using PLMs is to use a lexical (sparse) base [1, 6, 10, 11, 13, 27, 35, 49]. These systems take advantage of the PLMs to perform document and (sometimes) query expansion while also doing term-reweighting. Among the previously cited sparse models, SPLADE [10] has shown the best performance in out-of-domain tasks. However, it was originally presented with the same multi-cpu techniques than the dense models, which leads actually to large latencies on mono-cpu conditions [25]. We thus use SPLADE as the basis for this work, where we aim to improve its efficiency on mono-cpu retrieval. In the following, we do a quick summary on SPLADE.

SPLADE: It uses the BERT [7] token space to predict term importance ($|V| \approx 30k$). These term importances are based on the logits of the Masked Language Model (MLM). Let d a document and q a query, and let w_{ij} be the logit for i th token in d for the probability of term j . In other words, weight w_{ij} is how important the PLM considered the term j of the token space to the input token i . SPLADE then takes the importance for each token in the document sequence

and max pools them, to generate a vector in the BERT vocabulary space. SPLADE models are then optimized via distillation. These models jointly optimize: i) distance between teacher and student scores, and ii) minimize the expected mean FLOPS of the retrieval system. This joint optimization can be described as:

$$\mathcal{L} = \mathcal{L}_{distillation} + \lambda_q \mathcal{L}_{FLOPS}^q + \lambda_d \mathcal{L}_{FLOPS}^d \quad (1)$$

where $\mathcal{L}_{FLOPS}$ is the sparse FLOPS regularization from [36] and $\mathcal{L}_{distillation}$ is a distillation loss between the scores of a teacher and a student (in this work we use KL Divergence [14, 21] as the loss and a cross-ranker [33] as teacher). Note that there are two distinct regularization weights, so that one can put more sparsity pressure in either queries or documents, but always considering the amount of FLOPS.

SPLADE-doc: In [10] the authors also propose to consider a document-only version of SPLADE, i.e without query encoder. In this case, there is no query expansion, only tokenization is done, with all query terms having the same importance.

3 METHODS

Analyzing the setup from the multi-cpu retrieval used in SPLADEv2 [10] and the mono-cpu in [25], we derive that the main source of improvement is the reduction of SPLADE query sizes¹, instead of focusing solely on the FLOPS measure. The reason is that in mono-threaded systems, there are many techniques that allow for reducing the amount of effective FLOPS computed per query, but query size is then a major bottleneck. To get faster and more effective SPLADE models, we propose the following improvements: i) Searching for appropriate hyperparameters, ii) Improving data used for training, iii) Separating document and query encoders, iv) Changing query regularization to L1, v) Middle training of a PLM with a FLOPS regularization, and vi) Smaller PLM query encoder.

Of those adaptations, i) and ii) are just used in order to find better baselines. On the other hand, iii), iv), v) and vi) are novel contributions. Note that, we consider *each adaptation sequentially*, so that for example vi) is the system with all the proposed adaptations.

3.1 Baselines

i) Searching for appropriate hyperparameters: First of all, we checked if we could get more efficient networks with SPLADE as we change some training hyperparameters. Notably, we change the distillation loss from MarginMSE [17] to the more classical KL Divergence [14, 21] loss as in our studies it was more stable. Then, we also search for a set of (λ_q, λ_d) in order to have controlled query and document sizes. At the end of that experiment, we chose to keep a set of three configuration: **Small**, **Medium**, **Large**, where (S) is the sparsest configuration, (M) has the same query sparsity as S, but larger documents and (L) has larger queries than S and the same document sparsity as M.

ii) Improving data used for training: The second improvement comes from changing the data and model used for distillation. The goal is to improve the effectiveness of the networks chosen on the previous experiment, while avoiding increasing the cost of inference. To do so, we move from the more traditional set of

¹amount of tokens at the SPLADE output

distillation data from [17] to a newer one available from huggingface². The main difference is while the first one uses negatives from BM25, the latter uses negatives coming from various sources (dense models) and a more powerful teacher (trained with distillation).

3.2 Efficient SPLADE Models

With better baselines for SPLADE, let us introduce some changes to the overall model in order to improve its efficiency:

iii) Separating document and query encoders: While searching for proper baselines, we noticed that it was hard to achieve smaller queries, as even very large differences in λ_d and λ_q did not produce smaller queries. This is because the encoder for both documents and queries is the same and there is nothing to differentiate between them. Therefore we propose to have two distinct encoders for queries and documents, thus relieving the model to find an optimal trade-off for document and queries with a single model.

iv) L1 Regularization for Queries: A second change is to reconsider the FLOPS measure. While it make sense for document representation, it may not be the best measure accounting for latency of retrieval system. This is why we propose to change the regularization type of our query encoder to a simple L1 loss rather than the FLOPs on the query vectors.

v) PLM Middle Training with FLOPS regularization: Recently there has been a surge [12, 18] in what we call “middle-training” of PLMs for IR. The idea is that before fine-tuning on a dataset, one should refine a pre-trained model on the data and task that it will perform, with the most known example being Co-Condenser [12] for dense networks, that performs two steps of middle training: i) condensing information into the CLS with MLM ii) training jointly an unsupervised contrastive and MLM losses [7]. In this work, we investigate if performing a middle-training step, where we use an MLM task with a FLOPS regularization on the MLM logits, improves the final fine-tuned SPLADE model.

vi) Smaller PLM query encoder: An important factor when computing the latency of PLM-based models is the latency of the query encoder. This could be indeed very costly considering that we are on the mono-threaded, no-GPU configuration. In the SPLADEv2 paper [10] the authors propose to use a scheme without any query encoder (called SPLADE-doc). In this work, we evaluate two methods: i) remove the stop words of the queries and retrain SPLADE-doc model to further improve this method (we note by [†] the systems that remove the stop words of queries), and ii) use a very efficient PLM on the query encoder, namely BERT-tiny [3]³.

4 EXPERIMENTAL SETTING AND RESULTS

We trained and evaluated our models on the MS MARCO passage ranking dataset [2] in the full ranking setting. The dataset contains approximately 8.8M passages, and hundreds of thousands training queries with shallow annotation (≈ 1.1 relevant passages per query in average). The development set contains 6980 queries with similar

labels. We also consider the full BEIR benchmark [41] (18 datasets) which judges the zero-shot performance of IR models over diverse set of tasks and domains.

Measuring efficiency: In order to compute our efficiency numbers, all experiments are performed in the same machine, with an Intel(R) Xeon(R) Gold 6338 CPU @ 2.00GHz CPU and sufficient RAM memory to preload indexes, models and queries on memory before starting the experiment. Experiments are performed using Anserini and PISA⁴ for retrieval (following instructions obtained by contacting the authors of [25]) and PyTorch for document/query encoding. All efficiency experiments with PyTorch use the benchmarking tool from the transformers library [45].

The latency of query encoder PLMs was measured using a sequence length of 8 for average latency and 32 for 99 percentile latency. Latency is computed as a simple addition of query encoding and retrieval time. The DistilBERT query encoder has an average latency of 45.3ms and a 99 percentile latency of 57.6ms, while the BERT-tiny [3] query encoder has an average latency of 0.7ms and a 99 percentile latency of 1.1ms.

SPLADE Training: We use DistilBERT-base as the starting point for most models, safe for the improvements v) and vi), which use a middle-trained DistilBERT and middle-trained BERT-tiny. SPLADE Models are trained for 250k steps with the ADAM optimizer, using a learning rate of $2e^{-5}$ with linear scheduling and a warmup of 6000 steps, and a batch size of 128. We keep the last step as our final checkpoint. For the SPLADE-doc approach, we follow [10] and perform a reduced training of only 50k steps. We consider a maximum length of 256 for input sequences. In order to mitigate the contribution of the regularizer at the early stages of training, we follow [36] and use a scheduler for λ , quadratically increasing λ at each training iteration, until a given step (50k for SPLADE and 10k for SPLADE-doc), from which it remains constant. Middle-training is performed using default MLM parameters from [44], with an added FLOPS regularization [36] of $\lambda = 0.001$. Concerning (λ_q, λ_d) , models i), ii), iii) use the same hyperparameters: $S = (0.1, 5e - 3)$, $M = (0.1, 5e - 4)$, and $L = (0.01, 5e - 4)$, while models iv), v) and vi) use $S = (5e - 3, 5e - 3)$, $M = (5e - 4, 5e - 4)$, $L = (5e - 4, 5e - 4)$.

RQ1: How does the performance (efficiency and effectiveness) of SPLADE evolve as we introduce each subsequent adaptation? First we verify the relevance of adding each subsequent adaptation. Note that each “level” [i), ii), iii)...] **represents its change and all the others that came before**. We represent the efficiency and effectiveness of the systems in Figure 1, using pytorch for PLM inference and PISA for mono-cpu retrieval. For each model we have three points, representing each combination of λ_d, λ_q that we introduced in the previous section. SPLADEv2-distil is not shown on the Figure because it made it hard to read, with a reported latency of 265 [25] and a total latency measured on our system of 691 ms.

First, we see that just using a stronger baseline allows us to decrease the latency in PISA by almost 10 times (same in Anserini) of the latency of SPLADEv2-distil, while keeping similar or even improved performance on MS MARCO. This is expected, because

²in <https://huggingface.co/datasets/sentence-transformers/msmarco-hard-negatives>

³Another possibility would be to perform quantization and/or compression of the PLM, but we leave exploring this to future work

⁴Due to lack of space we only present latency PISA results on the figures and tables.

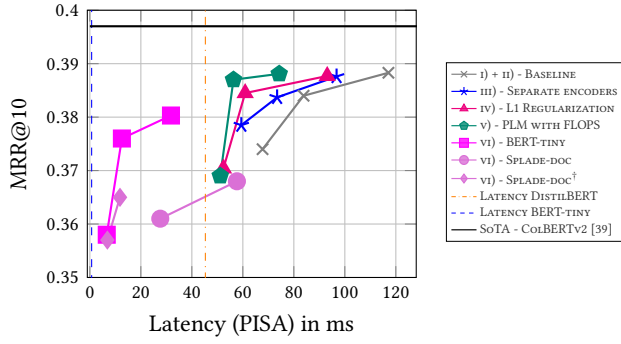


Figure 1: Latency comparison between all proposed improvements. †: queries without stop words.

the model used for comparison in [25] was not optimized for latency. We see also that all models trained are close to the single-stage state-of-the-art retrieval performance of ColBERTv2 [39] (at most a 10% reduction in MRR@10 performance).

Second, we see that improvements iii) through v) each successfully improve latency, while keeping similar effectiveness on MS MARCO. Note however, that each one seems to bring diminishing gains, because they are heavily impacted by the inference latency of DistilBERT, especially on PISA, for example, sparsest v) has a gain of 1.2ms compared to sparsest iv), which represents an approximate 20% reduction retrieval time (PISA), but an overall reduction of just 2% (PISA+Pytorch). In order to mitigate this, we introduce vi) which aims at speeding up the query encoder.

With vi) we note a trade-off, that at the cost of a slight effectiveness loss (≈ 1.0 MRR@10 on MS MARCO), we are able to greatly reduce the latency of the sparsest SPLADE models we test (≈ 10 fold PISA, ≈ 2 fold Anserini). For the query-encoder choice, BERT-tiny had a slight advantage over SPLADE-doc, showing the importance of a query-encoder, even if it is a very small one.

RQ2: How do these new SPLADE models compare to the state of the art in terms of in-domain sparse retrieval? In the previous question, we verified that the proposed improvements made sense in the context of SPLADE, but what does that mean against other methods in sparse retrieval? To answer this question we compare a subset of our systems {v) and vi)} to the baselines used in [25], namely: i) BM25, ii) DocT5 [34], iii) DeepImpact [27], and iv) uniCOIL-Tilde [50]. All methods are evaluated on our machine and DistilBERT latency is added to uniCOIL-Tilde. Results are displayed in Figure 2.

We can see that compared to the non-BM25 techniques we can achieve systems that are both more efficient and have better effectiveness for in-domain sparse retrieval⁵. Finally, compared to BM25 we achieve similar efficiency, with a 2x gain on effectiveness.

RQ3: How do these new SPLADE models compare to the state of the art in terms of out-of-domain sparse retrieval? In the previous sections we showed the efficiency and effectiveness of the proposed adaptations of SPLADE in-domain, i.e. on MS MARCO.

⁵Note that some of the improvements proposed here could be applied to these systems, but this is outside of the scope of our work, which focuses on SPLADE.

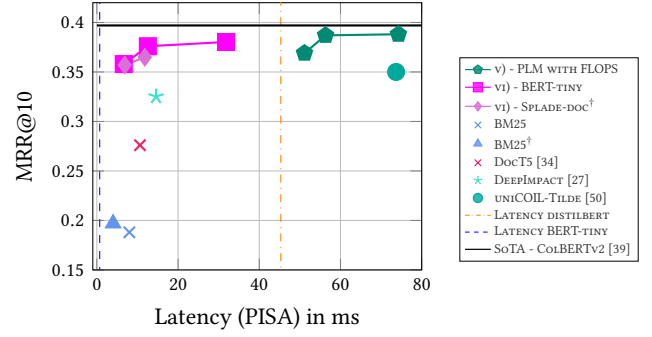


Figure 2: Latency comparison between the proposed adaptations and sparse methods. †: queries without stop words.

However, one of the main advantages of the SPLADEv2-distil model was in out-of-domain retrieval, notably on the BEIR benchmark (at the time of submission SoTA on the online benchmark). We now study the effect of our improvements, with MS MARCO MRR@10 and BEIR mean nDCG@10 results shown in Table 1⁶.

Table 1: Results on out-of-domain data and MS MARCO efficiency. Results for MSMARCO are given as MRR@10 on the 6980 dev set queries and nDCG@10 over the TREC-19 [5] queries, while BEIR results are the average nDCG@10. BEIR* is the combination of BM25 and the row’s method. BT: BERT-tiny query encoder. †: queries without stop-words. §: Differs from [10] as we consider the 18 beir datasets.

Method	Latency	MSMARCO	TREC19	BEIR	BEIR*
Baselines					
BM25 [†]	4	19.7	50.6	42.5	-
DocT5 [34]	11	27.6	64.2	43.4	-
SPLADEv2-distil [10]	691	36.8	72.9	47.0 [§]	49.3
Proposed models					
VI) BT-SPLADE-S	7	35.8	67.2	39.2	45.7
VI) BT-SPLADE-M	13	<u>37.6</u>	69.4	42.1	47.0
VI) BT-SPLADE-L	32	38.0	<u>70.3</u>	<u>44.5</u>	<u>47.9</u>

Unfortunately, the gains in efficiency and in-domain effectiveness seem to come at a cost of reducing the performance of the models outside of the MS MARCO domain. While it still has decent effectiveness compared to BM25 (which is not the case for most dense models), it still loses a lot of performance when compared to SPLADEv2-distil. Investigating the performances per dataset we noticed some sharp losses on some datasets, especially on the QUORA dataset (where nDCG@10 falls from 0.84 on SPLADEv2-distil to 0.46 on BT-SPLADE-S), which is expected as it uses queries (questions) as both documents and queries.

One way to mitigate this overall loss of effectiveness is by merging the document scores of the proposed models with the ones obtained by BM25. This comes at a cost, either adding the latency of BM25 (4ms) or by duplicating the computing cost but keeping the latency of the slowest model. We use a simple score combination

⁶A detailed per dataset result table is available in the Appendix.

inspired by [23], where documents not present in the top-100 of a model are assigned its smallest score and then normalizing the scores based on the maximum and minimum document scores of the method. Finally we simply sum the scores of the two methods, assigning equal weight to both. We represent the ensemble as column BEIR* on the table. We see that it allows us to slightly outperform SPLADEv2-distil by itself, while running under 40 ms of latency on a single cpu-core. On the more efficient models, combining our method with BM25 allows us to outperform DocT5 on BEIR on similar latency (11ms).

Finally, we also note that SPLADEv2-distil combined with BM25 is able to outperform methods that came after it [31, 32, 47] and had claimed the “state-of-the-art”. In other words, it is, as far as we are aware, the best result available on BEIR (49.3 mean on all 18 datasets).

5 CONCLUSION

In this paper we investigated the efficiency of SPLADE models and proposed small adaptations that led to more efficient systems. We demonstrated that the proposed adaptations improved both the efficiency and effectiveness of SPLADE for in-domain performance and that the small degradation on out-of-domain performance can be mitigated by combining with traditional sparse retrieval techniques. We also note that some of these adaptations could be applied to other systems, but that comparison is left as future work. In summary, to our knowledge, we propose the first neural models that *achieve similar mono-cpu latency and multi-cpu throughput as traditional BM25*, while having *similar performance* than state-of-the-art first-stage neural rankers on in-domain data (MS MARCO) and comparable performance on the (BEIR) benchmark to both BM25 and to most dense first-stage neural rankers.

ACKNOWLEDGMENTS

We would like to thank Thibault Formal, Hervé Dejean, Jean-Michel Renders and Simon Lupart for helping with their thoughtful comments. We also would like to thank the authors of Wacky Weights [25] (Joel Mackenzie, Andrew Trotman and Jimmy Lin) for sharing their PISA evaluation code with us.

REFERENCES

- [1] Yang Bai, Xiaoguang Li, Gang Wang, Chaoliang Zhang, Lifeng Shang, Jun Xu, Zhaowei Wang, Fangshan Wang, and Qun Liu. 2020. SparTerm: Learning Term-based Sparse Representation for Fast Text Retrieval. *arXiv:2010.00768* [cs.IR]
- [2] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. In *InCoCo@NIPS*.
- [3] Prajwal Bhargava, Aleksandr Drozd, and Anna Rogers. 2021. Generalization in NLI: Ways (Not) To Go Beyond Simple Heuristics. *arXiv:2110.01518* [cs.CL]
- [4] Andrei Z Broder, David Carmel, Michael Herscovici, Aya Soffer, and Jason Zien. 2003. Efficient query evaluation using a two-level retrieval process. In *Proceedings of the twelfth international conference on Information and knowledge management*. 426–434.
- [5] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M Voorhees. 2020. Overview of the trec 2019 deep learning track. *arXiv preprint arXiv:2003.07820* (2020).
- [6] Zhuyun Dai and Jamie Callan. 2019. Context-Aware Sentence/Passage Term Importance Estimation For First Stage Retrieval. *arXiv:1910.10687* [cs.IR]
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR abs/1810.04805* (2018). *arXiv:1810.04805* <http://arxiv.org/abs/1810.04805>
- [8] Constantinos Dimopoulos, Sergey Nepomnyachiy, and Torsten Suel. 2013. Optimizing top-k document retrieval strategies for block-max indexes. In *Proceedings of the sixth ACM international conference on Web search and data mining*. 113–122.
- [9] Shuai Ding and Torsten Suel. 2011. Faster top-k document retrieval using block-max indexes. In *Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval*. 993–1002.
- [10] Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval. *CoRR abs/2109.10086* (2021). *arXiv:2109.10086* <https://arxiv.org/abs/2109.10086>
- [11] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Virtual Event, Canada) (SIGIR '21). Association for Computing Machinery, New York, NY, USA, 2288–2292. <https://doi.org/10.1145/3404835.3463098>
- [12] Luyu Gao and Jamie Callan. 2021. Unsupervised corpus aware language model pre-training for dense passage retrieval. *arXiv preprint arXiv:2108.05540* (2021).
- [13] Luyu Gao, Zhuyun Dai, and Jamie Callan. 2021. COIL: Revisit Exact Lexical Match in Information Retrieval with Contextualized Inverted List. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 3030–3042. <https://doi.org/10.18653/v1/2021.naacl-main.241>
- [14] Geoffrey Hinton, Oriol Vinyals, Jeff Dean, et al. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* 2, 7 (2015).
- [15] Sebastian Hofstätter and Allan Hanbury. 2019. Let’s measure run time! Extending the IR replicability infrastructure to include performance aspects. *SIGIR Open-Source IR Replicability Challenge (OSIRRC)* (2019).
- [16] Sebastian Hofstätter, Sheng-Chieh Lin, Jheng-Hong Yang, Jimmy Lin, and Allan Hanbury. 2021. Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling. In *Proc. of SIGIR*.
- [17] Sebastian Hofstätter, Sophia Althammer, Michael Schröder, Mete Sertkan, and Allan Hanbury. 2020. Improving Efficient Neural Ranking Models with Cross-Architecture Knowledge Distillation. *arXiv:2010.02666* [cs.IR]
- [18] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Towards Unsupervised Dense Information Retrieval with Contrastive Learning. *arXiv preprint arXiv:2112.09118* (2021).
- [19] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [20] Omar Khattab, Mohammad Hammoud, and Tamer Elsayed. 2020. Finding the best of both worlds: Faster and more robust top-k document retrieval. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1031–1040.
- [21] Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. *The annals of mathematical statistics* 22, 1 (1951), 79–86.
- [22] Jimmy Lin and Andrew Trotman. 2015. Anytime ranking for impact-ordered indexes. In *Proceedings of the 2015 International Conference on The Theory of Information Retrieval*. 301–304.
- [23] Sheng-Chieh Lin, Jheng-Hong Yang, and Jimmy Lin. 2021. In-Batch Negatives for Knowledge Distillation with Tightly-Coupled Teachers for Dense Retrieval. In *Proceedings of the 6th Workshop on Representation Learning for NLP (Repl4NLP-2021)*. Association for Computational Linguistics, Online, 163–173. <https://doi.org/10.18653/v1/2021.repl4nlp.1.17>
- [24] Joel Mackenzie, Zhuyun Dai, Luke Gallagher, and Jamie Callan. 2020. Efficiency implications of term weighting for passage retrieval. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1821–1824.
- [25] Joel Mackenzie, Andrew Trotman, and Jimmy Lin. 2021. Wacky weights in learned sparse representations and the revenge of score-at-a-time query evaluation. *arXiv preprint arXiv:2110.11540* (2021).
- [26] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [27] Antonio Mallia, Omar Khattab, Torsten Suel, and Nicola Tonellotto. 2021. Learning Passage Impacts for Inverted Indexes. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Virtual Event, Canada) (SIGIR '21). Association for Computing Machinery, New York, NY, USA, 1723–1727. <https://doi.org/10.1145/3404835.3463030>
- [28] Antonio Mallia, Giuseppe Ottaviano, Elia Porciani, Nicola Tonellotto, and Rossano Venturini. 2017. Faster BlockMax WAND with variable-sized blocks. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 625–634.
- [29] Antonio Mallia, Michal Siedlaczek, Joel Mackenzie, and Torsten Suel. 2019. PISA: Performant indexes and search for academia. *Proceedings of the Open-Source IR Replicability Challenge* (2019).
- [30] Antonio Mallia, Michal Siedlaczek, and Torsten Suel. 2021. Fast Disjunctive Candidate Generation Using Live Block Filtering. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. 671–679.

- [31] Niklas Muennighoff. 2022. SGPT: GPT Sentence Embeddings for Semantic Search. *arXiv preprint arXiv:2202.08904* (2022).
- [32] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. 2022. Text and Code Embeddings by Contrastive Pre-Training. *arXiv preprint arXiv:2201.10005* (2022).
- [33] Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage Re-ranking with BERT. *arXiv:1901.04085* [cs.IR]
- [34] Rodrigo Nogueira and Jimmy Lin. 2019. From doc2query to docTTTTTquery. Rodrigo Nogueira, Wei Yang, Jimmy Lin, and Kyunghyun Cho. 2019. Document Expansion by Query Prediction. *arXiv:1904.08375* [cs.IR]
- [35] Biswajit Paria, Chih-Kuan Yeh, Ian E. H. Yen, Ning Xu, Pradeep Ravikumar, and Barnabás Póczos. 2020. Minimizing FLOPs to Learn Efficient Sparse Representations. *arXiv:2004.05665* [cs.LG]
- [36] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 5835–5847. <https://doi.org/10.18653/v1/2021-naacl-main.466>
- [37] S. Robertson. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389. http://scholar.google.de/scholar.bib?q=info:U4l9kCVIssAJ:scholar.google.com/&output=citation&hl=de&as_sdt=2000&as_vis=1&ct=citation&cd=1
- [38] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2021. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. *arXiv preprint arXiv:2112.01488* (2021).
- [39] Josef Sivic and Andrew Zisserman. 2003. Video Google: A text retrieval approach to object matching in videos. In *Computer Vision, IEEE International Conference on*, Vol. 3. IEEE Computer Society, 1470–1470.
- [40] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. *CoRR* abs/2104.08663 (2021). *arXiv:2104.08663* <https://arxiv.org/abs/2104.08663>
- [41] Zhengkai Tu, Wei Yang, Zihang Fu, Yuqing Xie, Luchen Tan, Kun Xiong, Ming Li, and Jimmy Lin. 2020. Approximate Nearest Neighbor Search and Lightweight Dense Vector Reranking in Multi-Stage Retrieval Architectures. In *Proceedings of the 2020 ACM SIGIR on International Conference on Theory of Information Retrieval*. 97–100.
- [42] Howard Turtle and James Flood. 1995. Query evaluation: strategies and optimizations. *Information Processing & Management* 31, 6 (1995), 831–850.
- [43] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *arXiv:1910.03771* [cs.CL]
- [44] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45. <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [45] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=zeFrfgYzln>
- [46] Canwen Xu, Daya Guo, Nan Duan, and Julian McAuley. 2022. LaPraDoR: Unsupervised Pretrained Dense Retriever for Zero-Shot Text Retrieval. *arXiv preprint arXiv:2203.06169* (2022).
- [47] Peilin Yang, Hui Fang, and Jimmy Lin. 2018. Anserini: Reproducible ranking baselines using Lucene. *Journal of Data and Information Quality (JDIQ)* 10, 4 (2018), 1–20.
- [48] Tiancheng Zhao, Xiaopeng Lu, and Kyusong Lee. 2020. SPARTA: Efficient Open-Domain Question Answering via Sparse Transformer Matching Retrieval. *arXiv:2009.13013* [cs.CL]
- [49] Shengyao Zhuang and Guido Zuccon. 2021. Fast passage re-ranking with contextualized exact term matching and efficient passage expansion. *arXiv preprint arXiv:2108.08513* (2021).

A COMPARISON AGAINST DENSE MODELS

In the main paper we compare mostly against sparse models and only report the effectiveness of a SoTA dense multi-representation model (ColBERTv2 [39]). We do so because we do not feel there is a proper way to compare both methods (for example all our latency numbers use PISA for retrieval and Pytorch for inference, which is impossible in the case of dense models). For completeness in this section we investigate a comparison between a SoTA mono-representation dense model (CoCondenser [12] available at <https://huggingface.co/Luyu/co-condenser-marco-retriever>) and the models studied in this work.

In order to study different settings that make different trade-offs we define three: i) Measuring latency under mono-cpu using one query at a time, ii) Measuring QPS (queries per second) under multi-threaded-cpu using batched queries, and iii) Measuring QPS (queries per second) multi-threaded-cpu disregarding inference time with batched queries. The dense model uses the FAISS [19] framework, where we either do brute-force or perform ANN using HNSW [26] and IVF [40] indexes, without any quantization⁷. While we don't feel that any of the three settings are fully fair to both approaches (they fail to acknowledge index size, maximum RAM and ANN use for sparse retrieval [25, 42]), we feel that overall they represent some of the different scenarios that one would consider for deploying such systems.

Also note that the dense model already starts at a disadvantage due to the fact that it uses a larger encoder (BERT, around two times the computational cost of distilBERT) and that as far as we are aware there are no SoTA dense implementations using BERT-tiny. In order to consider a distilBERT version of CoCondenser we assume a *perfect* quantization of the BERT model into distilBERT and a scenario where inference is disregarded, which are not realistic, but give an idea of the best possible numbers are if we had "perfect" inference. Results are depicted in Figures 3, 4, and 5.

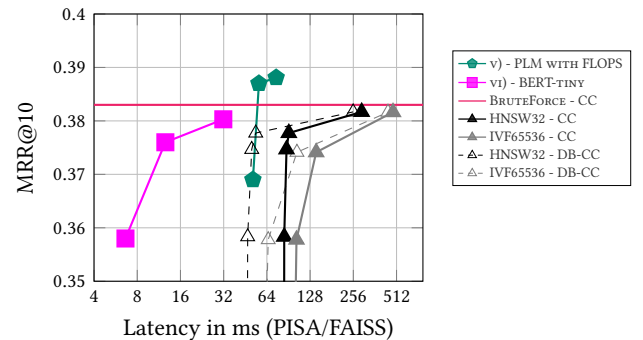
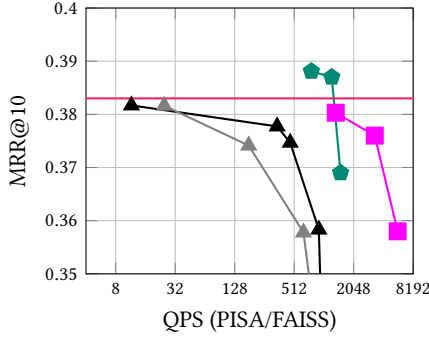


Figure 3: Latency comparison between the proposed adaptations and dense methods with ANN. Results are better up and to the left. axis in $\log_2$ scale. CC: CoCondenser. DB CoCondenser is an ideal quantization of CoCondenser from BERT to distilBERT.

⁷We tested with product quantization and the efficiency-effectiveness trade-off was worse than without. As we do not show index size on the figures we preferred to only deal with non quantized models in order to reduce the number of parameters to tune

Table 2: Detailed BEIR results using 100 times the nDCG@10 for each dataset. BT: BERT-tiny query encoder and BT-S/M/L is the VI) BT-SPLADE-S/M/L model. †: model uses queries without stop-words. Bold represents best in row.

Dataset	Baselines					Proposed					
	BM25 [†]	BM25	DocT5	SPLADEv2-distil	SPLADEv2-distil+BM25 [†]	BT-S	BT-M	BT-L	BT-S + BM25 [†]	BT-M + BM25 [†]	BT-L + BM25 [†]
arguana	32.00	31.55	34.90	47.91	47.98	45.47	46.58	47.29	47.28	47.23	48.17
bioasq	47.67	46.46	43.10	50.80	55.22	41.17	45.20	47.13	50.69	53.09	53.72
climate-fever	21.32	21.29	20.10	23.53	26.59	17.09	18.00	18.89	22.81	23.34	23.58
cqadupstack	28.53	29.87	32.50	35.01	34.49	28.22	30.52	33.01	31.35	32.53	33.70
dbpedia-entity	32.26	31.28	33.10	43.50	41.01	35.38	38.91	40.54	37.99	39.35	40.09
fever	74.35	75.31	71.40	78.62	82.44	69.88	72.14	74.87	79.13	79.29	80.04
fiqa	24.30	23.61	29.10	33.61	32.45	28.78	30.07	31.77	30.89	31.15	31.80
hotpotqa	60.13	60.28	58.00	68.44	70.02	56.59	61.31	66.61	63.80	66.34	68.44
nfcopus	32.67	32.55	32.80	33.43	34.07	31.52	32.43	33.10	33.16	33.50	33.70
nq	32.87	32.86	39.90	52.08	48.20	47.35	49.69	51.48	46.25	47.24	48.33
quora	74.71	78.86	80.20	83.76	84.99	46.29	62.24	72.34	68.45	74.86	78.54
robust04	41.91	40.84	43.70	46.75	49.52	33.73	37.07	40.66	44.02	45.64	46.60
scidocs	15.83	15.81	16.20	15.79	16.92	14.56	14.56	15.25	16.43	16.47	16.80
scifact	66.28	66.47	67.50	69.25	71.58	63.94	65.77	67.43	68.66	69.59	70.45
signal1m	32.69	33.05	30.70	26.56	33.31	24.50	27.21	28.28	31.56	33.47	33.51
trec-covid	71.23	65.59	71.30	71.04	76.89	59.91	64.38	66.06	73.67	75.64	75.63
trec-news	40.33	39.77	42.00	39.18	45.46	35.43	34.80	38.65	43.30	43.03	43.96
webis-touche2020	35.40	36.73	34.70	27.18	35.85	26.54	26.42	27.03	33.53	34.89	35.39
Average	42.47	42.34	43.40	47.02	49.28	39.24	42.07	44.47	45.72	47.04	47.91
Best on	0	1	0	4	11	0	0	0	0	0	2

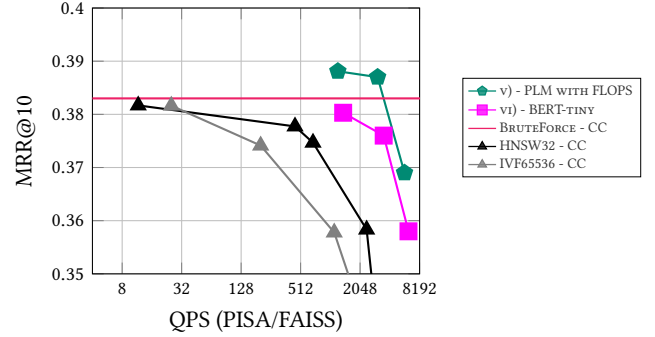
**Figure 4: QPS comparison between the proposed adaptations and dense methods with ANN, considering inference time. Results are better up and to the right. axis in $\log_4$ scale. DB-CoCondenser omitted as difference with CoCondenser is too small. CC: CoCodenser**

In mono-cpu latency the brute force dense approach takes too long compared to all others (more than 200 ms), so we do not depict it on the image. We thus have to compare our sparse retrieval methods with ANN-based dense retrieval⁸. We were actually surprised that HNSW allows dense models to be very close to the efficient SPLADE models (if we consider perfect quantization from BERT to distilBERT), however, HNSW comes with a drawback of increased index size: in our case 256 bytes extra per passage or around 2GB for MSMARCO⁹. If we consider methods that do not increase index size by much (such as IVF) the difference is still pretty large¹⁰. Under multiple cpu threads we still see an advantage for the models

⁸We could also apply ANN to sparse retrieval [25, 42], but we leave this as future work

⁹The entire SPLADE pisa index takes around 1GB for S, 2GB for M and 4GB for L

¹⁰Unquantized dense indexes are around 25GB for fp32 and 12GB for fp16

**Figure 5: QPS comparison between the proposed adaptations and dense methods with ANN, disregarding inference time. Results are better up and to the right. axis in $\log_4$ scale. CC: CoCodenser.**

presented in this work, even when compared to BERT-based CoCodenser (Fig 4) or even if we disregard inference time (Fig 5). These results further show the interest of studying sparse representations (equal/better efficiency-effectiveness trade-off with smaller index).

B DETAILED BEIR RESULTS

For completeness, we now present the BEIR results, dataset per dataset, in Table 2. We can see that SPLADEv2-distil actually dominates all other approaches in the amount of datasets that it is the best on (4 by itself and 11 alongside BM25) and that BM25 by itself is only the best in webis-touche2020. The combination of VI)BT-SPLADE-M and L with BM25 present more balanced results, having less datasets that they outperform the others compared with SPLADEv2-distil, however they have a similar/better average effectiveness for less than 5% the cost of running SPLADEv2-distil.